

ОТЧЕТ
по лабораторной работе № 8
по дисциплине «Программирование на Python»

Тема: Клиент-серверное приложение на Python с использованием Jinja2

Выполнил: студент гр. Р3121
Баранова Елизавета Павловна

Преподаватель: Жуков Николай
Николаевич

1. Цели работы

- Создать простое клиент-серверное приложение на Python без использования серверных фреймворков.
- Освоить работу с HTTPServer и реализовать маршрутизацию HTTP-запросов.
- Применять шаблонизатор Jinja2 для отображения данных в HTML.
- Реализовать модели предметной области (User, Currency, UserCurrency, App, Author) с геттерами и сеттерами для проверки корректности данных.
- Структурировать код в соответствии с архитектурой MVC (модель–вид–контроллер).
- Получать актуальные данные о курсах валют через функцию `get_currencies` и отображать их пользователям.
- Реализовать функциональность подписки пользователей на валюты и возможность отображения динамики их изменения.
- Научиться создавать тесты для моделей и серверной логики, проверяя корректность работы приложения.

2. Описание предметной области

Приложение моделирует систему управления пользователями и их подписками на валюты:

Author — автор приложения, свойства: `name`, `group`.

App — приложение, свойства: `name`, `version`, `author` (связь с объектом Author).

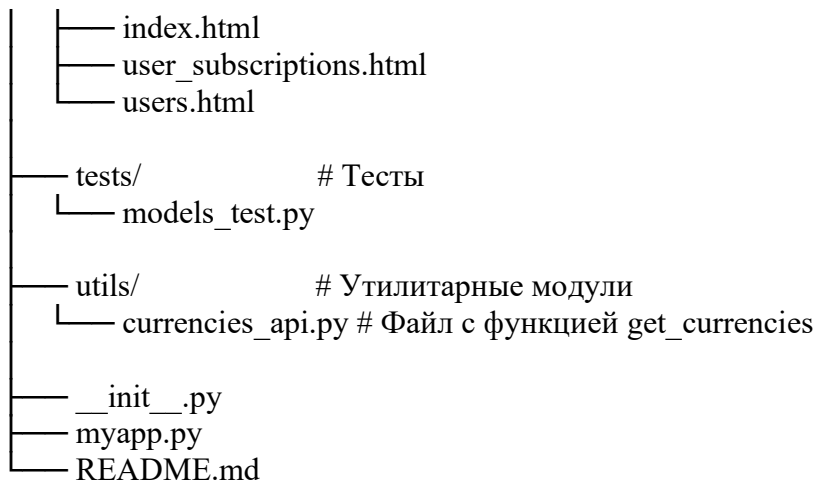
User — пользователь, свойства: `id`, `name`.

Currency — валюта, свойства: `id`, `num_code`, `char_code`, `name`, `value`, `nominal`.

UserCurrency — связь пользователя с валютой, свойства: `id`, `user_id`, `currency_id`.

3. Структура проекта

```
lab8/
├── models/                    # Модели приложения
│   ├── __init__.py
│   ├── app.py
│   ├── author.py
│   ├── currency.py
│   ├── user.py
│   └── user_currency.py
├── static/
├── templates/                # HTML-шаблоны
│   ├── author.html
│   └── currencies.html
```



4. Описание реализации

Реализация моделей и их свойств

Все модели реализованы как классы с приватными атрибутами.
Для каждого свойства реализованы геттеры и сеттеры, которые:

- Возвращают текущее значение атрибута через геттер.
- Позволяют безопасно изменять значение через сеттер с проверкой типа, длины или диапазона.
- Генерируют ошибки при некорректных данных.

Реализация маршрутов и обработки запросов

Использован модуль `http.server` и класс `BaseHTTPRequestHandler`.

Основная логика в методе `do_GET`:

`/` — главная страница.
`/users` — список всех пользователей.
`/users/<id>` — подписки конкретного пользователя.
`/currencies` — список валют и их курсов.
`/author` — информация об авторе.

Сервер отправляет HTML-страницу в зависимости от пути.

Использование шаблонизатора Jinja2

Инициализация окружения Jinja2 для загрузки шаблонов из пакета `myapp`:

```
from jinja2 import Environment, PackageLoader, select_autoescape
from http.server import HTTPServer, BaseHTTPRequestHandler
from utils.currencies_api import get_currencies

# Настройка шаблонизатора Jinja2
env = Environment(
    loader=PackageLoader("myapp"),
    autoescape=select_autoescape()
)
```

Для генерации HTML вызывается метод `render()` с передачей данных:

```

index_html = env.get_template("index.html").render(
    myapp="Приложение для отслеживания курсов валют",
    author_name='Liza',
    group='P3121',
    navigation=[
        {'caption': 'Главная', 'href': "/"},
        {'caption': 'Пользователи', 'href': "/users"},
        {'caption': 'Курсы валют', 'href': "/currencies"},
        {'caption': 'Об авторе проекта', 'href': "/author"}
    ]
)

```

Интеграция функции `get_currencies`

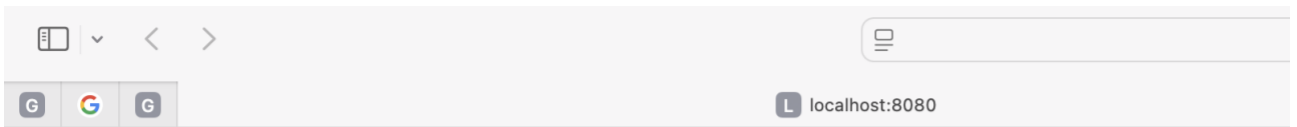
- Функция `get_currencies` получает актуальные курсы валют через API.
- Используется для формирования словаря `currencies`

```

currency_codes = ['USD', 'EUR', 'GBP']
currency_values = get_currencies(currency_codes)
currencies = {
    code: {
        'char_code': code,
        'name': {
            'USD': 'Доллар США',
            'EUR': 'Евро',
            'GBP': 'Фунт стерлингов'
        }[code],
        'value': currency_values[code]
    }
    for code in currency_codes
}

```

5. Примеры работы приложения



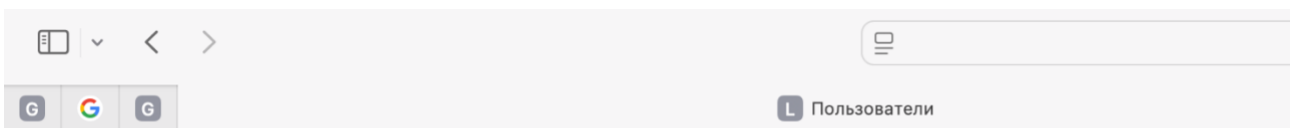
Приложение для отслеживания курсов валют

Автор проекта: Liza

Группа: P3121

- [Главная](#)
- [Пользователи](#)
- [Курсы валют](#)
- [Об авторе проекта](#)

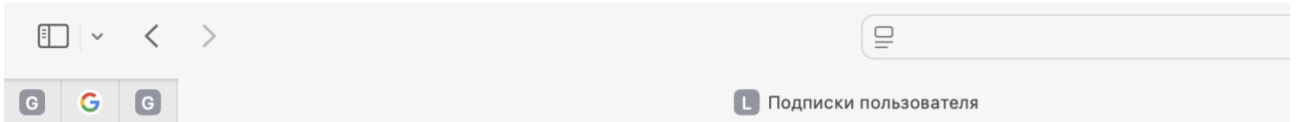
<



Список пользователей

- [Alexandr](#)
- [Vasiliy](#)

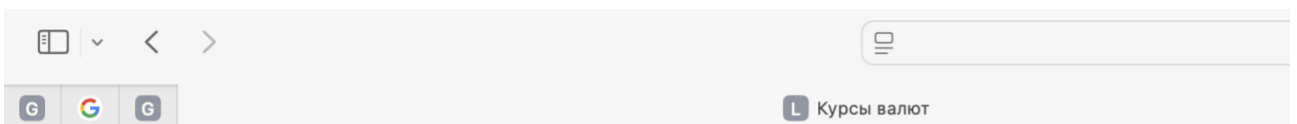
[Главная](#)



Подписки пользователя Alexandr

- USD — Доллар США: 76.0937
- EUR — Евро: 88.7028

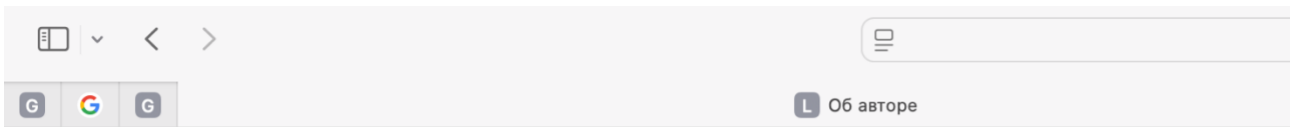
[Назад](#)



Список валют

- USD — Доллар США: 76.0937 (номинал 1)
- EUR — Евро: 88.7028 (номинал 1)
- GBP — Фунт стерлингов: 101.7601 (номинал 1)

[Главная](#)



Об авторе

Имя: Liza

Группа: P3121

[Главная](#)

```
Run  myapp x
Server is running on http://localhost:8080
127.0.0.1 - - [06/Dec/2025 13:43:42] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [06/Dec/2025 13:44:38] "GET /users HTTP/1.1" 200 -
127.0.0.1 - - [06/Dec/2025 13:44:48] "GET /users/1 HTTP/1.1" 200 -
127.0.0.1 - - [06/Dec/2025 13:45:01] "GET /currencies HTTP/1.1" 200 -
127.0.0.1 - - [06/Dec/2025 13:45:13] "GET /author HTTP/1.1" 200 -
```

6. Тестирование

Пример теста для модели

```
class TestAuthorModel(unittest.TestCase):
    """Тестирование модели Author."""
    def test_getters_setters(self):
        """Проверка работы геттеров и сеттеров."""
        author = Author("Liza", "P3121")
        self.assertEqual(author.name, "Liza")
        self.assertEqual(author.group, "P3121")
        author.name = "Anna"
        self.assertEqual(author.name, "Anna")
        author.group = "P1234"
        self.assertEqual(author.group, "P1234")

    def test_invalid_values(self):
        """Проверка обработки некорректных значений."""
        with self.assertRaises(ValueError):
            Author("A", "P3121")
        with self.assertRaises(ValueError):
            Author("Liza", "P1")
```

Примеры теста контроллера

```
class TestController(unittest.TestCase):
    """Тестирование HTTP-контроллера."""
    @classmethod
    def setUpClass(cls):
        """Запуск сервера в отдельном потоке для тестов."""
        cls.server = HTTPServer(('localhost', 8081), SimpleHTTPRequestHandler)
        cls.thread = threading.Thread(target=cls.server.serve_forever)
        cls.thread.daemon = True
        cls.thread.start()

    def get_path(self, path):
        """Функция для выполнения GET-запроса."""
        conn = http.client.HTTPConnection('localhost', 8081)
        conn.request("GET", path)
        response = conn.getresponse()
        data = response.read().decode()
        conn.close()
        return response.status, data

    def test_index(self):
        """Проверка главной страницы."""
        status, data = self.get_path("/")
        self.assertEqual(status, 200)
        self.assertIn("Приложение для отслеживания курсов валют", data)
```

Примеры теста функции get_currencies

```
class TestGetCurrenciesFunction(unittest.TestCase):
    """Тестирование функции получения курсов валют."""

    def test_valid_currencies(self):
        """Проверка корректного получения существующих валют."""
        result = get_currencies(['USD', 'EUR'])
        self.assertIn('USD', result)
        self.assertIn('EUR', result)
        self.assertIsInstance(result['USD'], float)

    def test_nonexistent_currency(self):
        """Проверка обработки несуществующей валюты."""
        with self.assertRaises(KeyError):
            get_currencies(['XYZ'])
```

7. Выводы

Проблемы при реализации:

- При работе с HTTPServer нужно было учитывать корректную обработку разных типов запросов и ошибок.
- Использование Jinja2 потребовало внимательного управления шаблонами, чтобы динамически формировать страницы без дублирования кода.

Применение принципов MVC:

- **Модель:** функции `get_currency` отвечала за получение и подготовку данных о курсах валют
- **Контроллер:** `HTTPServer` принимал запросы, вызывал функции из модели и передавал результаты в представление
- **Представление:** шаблоны `Jinja2` динамически отображали данные

Новые навыки:

- Работа с `HTTPServer`: настройка маршрутов, корректная обработка GET-запросов и ошибок.
- `Jinja2`: использование шаблонов, циклов, условий и контекста для динамической генерации HTML.